

Comparativa de Assembler en MIPS, ARM e Intel

22026

Juan Camilo Acevedo Bedoya | Facultad de Ingenierías

Subtítulo

Cuerpo del texto.
Se sugiere utilizar más imágenes que texto.
Se puede implementar tipografía Arial o Nimbus.

MIPS: La Filosofía RISC

Reduced Instruction Set Computer — hacer más con menos instrucciones, pero instrucciones simples y predecibles.

- Instrucciones de longitud fija: todas miden exactamente **32 bits**
- Modelo **Load/Store**: la única forma de acceder a memoria es con instrucciones dedicadas; el resto opera solo en registros
- **32 registros** de propósito general, nombrados con convenciones claras ($t0, $s0, $a0…)
- Ideal para educación: arquitectura ortogonal, sin excepciones extrañas

¿Por qué MIPS en el aula?

MIPS fue diseñado en Stanford con principios académicos. Su regularidad lo hace el laboratorio perfecto para entender cómo funciona cualquier CPU RISC moderna. ARM, RISC-V y muchos otros heredan sus ideas fundamentales.

Patterson & Hennessy — el libro de texto de arquitectura de computadoras más influyente del mundo — usa MIPS como arquitectura de referencia.

ARM: Eficiencia Global

📱 Omnipresencia

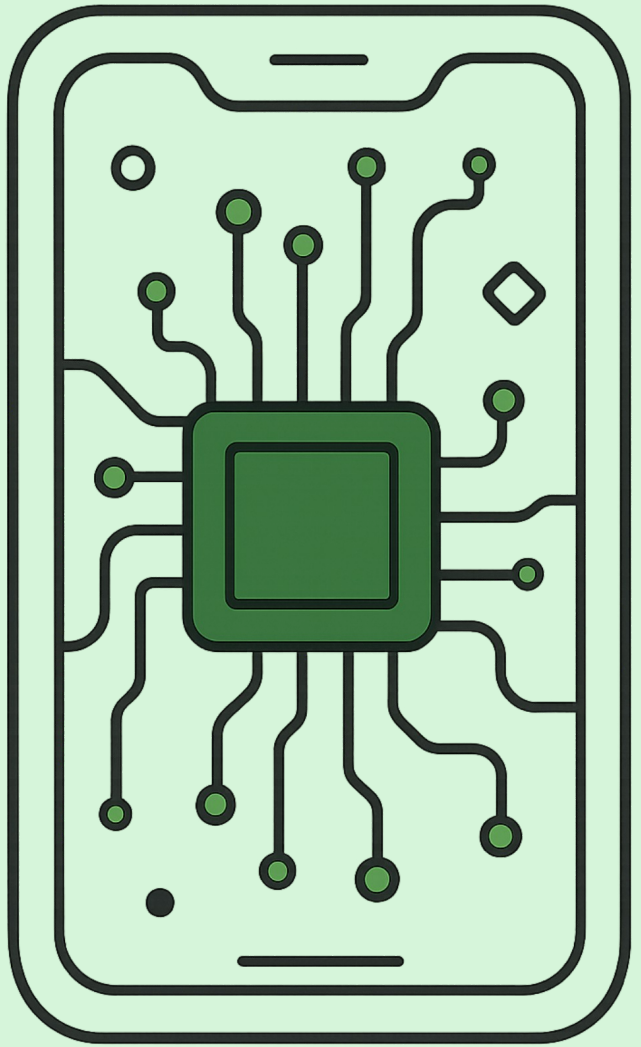
Más del 95% de los smartphones del mundo usan ARM. También domina tablets, IoT, y desde 2020, las laptops Apple (M1/M2/M3).

⚡ Bajo Consumo

Arquitectura RISC optimizada para extraer la máxima operación por miliamperio. La batería de tu celular dura horas gracias a estas decisiones de diseño.

🎯 Condicionalidad

Característica única: muchas instrucciones ARM pueden ejecutarse condicionalmente con sufijos (EQ, NE, GT…), reduciendo saltos y mejorando el pipeline.



Intel (x86): El Poder del CISC

Complex Instruction Set Computer — la filosofía opuesta: instrucciones poderosas que hacen más trabajo individualmente, con décadas de compatibilidad hacia atrás.

- **Instrucciones de longitud variable**: desde 1 hasta 15 bytes por instrucción
- Operaciones directas sobre **memoria**: no es obligatorio cargar a registro primero
- Solo **8 registros** generales en modo 32-bit (EAX, EBX, ECX…), extendidos a 16 en 64-bit
- Compatibilidad total con código de los años 80 hasta hoy

Internamente, los procesadores Intel modernos descomponen instrucciones CISC en micro-operaciones similares a RISC para ejecutarlas eficientemente.

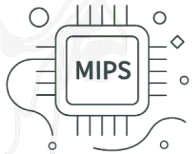
La paradoja Intel

x86 es CISC por fuera, RISC por dentro. El frontend decodifica instrucciones complejas; el backend las ejecuta como micro-ops simples en múltiples unidades paralelas.

Esta arquitectura híbrida es lo que permite a Intel mantener compatibilidad histórica sin sacrificar rendimiento moderno.

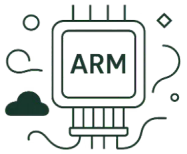
Caso de Estudio: Suma Simple (`a = b + c`)

La operación más básica de la programación. En C es una sola línea; en ensamblador revela las diferencias fundamentales entre arquitecturas.



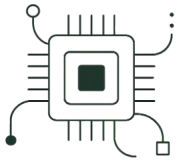
MIPS

Requiere que **b** y **c** estén en registros. Usa `add` con tres operandos: destino, fuente1, fuente2. Todo ocurre en registros.



ARM

Sintaxis muy similar a MIPS. Tres operandos en `ADD`. Los registros se nombran r0, r1, r2… La asignación del destino es flexible.



Intel x86

Solo dos operandos: el destino también es fuente. Requiere un `MOV` previo para no destruir el valor de **b** al sumar.

Comparativa: MIPS vs ARM vs Intel — Suma

El mismo resultado (`a = b + c`), tres formas de expresarlo en ensamblador. Observa cómo cambia la cantidad de instrucciones y el modelo de operandos.

Arquitectura	Instrucción Assembler	Explicación
C (alto nivel)	`a = b + c;`	Una sola expresión, sin revelar registros ni memoria
MIPS	`add $s0, $s1, $s2`	$s0 ← $s1 + $s2. Tres operandos explícitos, formato R-type
ARM	`ADD r0, r1, r2`	r0 ← r1 + r2. Sintaxis similar, convención de nombres diferente
Intel x86	`MOV eax, ebxADD eax, ecx`	eax ← ebx; luego eax ← eax + ecx. Dos instrucciones por restricción de operandos

MIPS y ARM permiten tres operandos distintos (destino ≠ fuentes). x86 en modo 32-bit solo tiene dos operandos: el destino es también una de las fuentes.

Acceso a Memoria: Cargando Datos

El cuello de botella clásico de toda arquitectura: mover bytes entre la RAM y los registros del procesador. Aquí las filosofías divergen claramente.

MIPS — `lw $t0, 0($s0)`

Load Word: carga 4 bytes desde la dirección `Mem[$s0 + 0]` hacia el registro `$t0`. El desplazamiento (offset) es inmediato. Para almacenar: `sw $t0, 0($s0)`. Solo estas instrucciones tocan la memoria.

ARM — `LDR r0, [r1]`

Load Register: carga desde la dirección almacenada en `r1` hacia `r0`. ARM soporta modos avanzados: `LDR r0, [r1, #4]` para offset, o `LDR r0, [r1, r2, LSL #2]` para índice escalado.

Intel x86 — `MOV eax, [ebx]`

MOV con direccionamiento indirecto: a diferencia de MIPS/ARM, x86 permite operar directamente sobre memoria sin carga previa. `ADD eax, [ebx]` suma el contenido de memoria al registro en una sola instrucción.

Estructuras de Control: El `if/else`

La lógica condicional es el corazón de cualquier programa. Cada arquitectura tiene su propia forma de evaluar condiciones y redirigir el flujo de ejecución.

MIPS

No hay flags de estado. La comparación se hace explícitamente con `slt` (Set Less Than), que escribe 1 o 0 en un registro. Luego `beq` o `bne` ramifica según ese valor.

```
slt $t0, $s0, $s1
bne $t0, $zero, then
```

ARM

Usa flags de estado (N, Z, C, V) actualizados por `CMP`. Las instrucciones pueden ejecutarse condicionalmente con sufijos: `ADDLT`, `SUBGE`, etc. Reduce la cantidad de saltos necesarios.

```
CMP r0, r1
ADDLT r2, r2, #1
```

Intel x86

`CMP` actualiza los flags del registro EFLAGS. Luego los saltos condicionales (`JE`, `JNE`, `JL`, `JG`) leen esos flags para decidir si saltar.

```
CMP eax, ebx
JL etiqueta_then
```

Ejemplo 1: El Bucle `for` Simple

El bucle `for (i=0; i<10; i++)` es trivial en C, pero en assembler expone la lógica del contador, la comparación y el salto hacia atrás.

Arq.	Secuencia de instrucciones	Notas pedagógicas
MIPS	`add $t0, $zero, $zeroloop: slt $t1, $t0, $s0beq $t1, $zero, fin... cuerpo ...addi $t0, $t0, 1j loop`	Inicializar contador, comparar con `slt`, saltar con `beq/j`. Cada paso es explícito y visible.
ARM	`MOV r0, #0loop: CMP r0, #10BGE fin... cuerpo ...ADD r0, r0, #1B loop`	La comparación y el salto son más directos gracias a los flags. `BGE` (Branch if Greater or Equal) simplifica el control.
Intel	`MOV ecx, 10loop: ... cuerpo ...LOOP loop`	`LOOP` decrementa `ecx` y salta si no es cero. Un bucle completo en una sola instrucción CISC.

Ejemplo 2: Manipulación de Arreglos (`array[i] = x`)

Acceder a un elemento de arreglo implica calcular su dirección en memoria. Cada elemento ocupa 4 bytes, por lo que el índice debe escalarse.

1

MIPS — Dirección explícita

Se calcula manualmente: `sll $t1, $t0, 2` (multiplica índice × 4), luego `add $t2, $s0, $t1` para la dirección base + offset. Finalmente `sw $s1, 0($t2)` almacena el valor. Tres pasos, todo visible.

2

ARM — Escalado en el operando

`STR r1, [r0, r2, LSL #2]` hace todo en una sola instrucción: base `r0`, índice `r2` escalado por 4 (LSL #2). ARM oculta la multiplicación dentro del modo de direccionamiento.

3

Intel — Direccionamiento complejo

`MOV [ebx + ecx*4], eax` usa el modo SIB (Scale-Index-Base) de x86. Base + índice × escala + desplazamiento, todo en un operando. Máximo poder expresivo CISC.

Ejemplo 3: Llamadas a Funciones

Invocar una función requiere guardar la dirección de retorno, pasar argumentos y gestionar la pila. Cada arquitectura tiene su propio protocolo.

MIPS — Registro $ra

`jal función` salta y guarda la dirección de retorno en `$ra`. Al terminar, `jr $ra` regresa. Para funciones anidadas, `$ra` debe guardarse en la pila con `sw $ra, 0($sp)`.

ARM — Link Register (LR)

`BL función` (Branch and Link) guarda la dirección de retorno en el registro `LR`. Para retornar: `MOV pc, lr` o simplemente `BX lr`. Funciones hoja no necesitan usar la pila.

Intel — CALL y RET

`CALL función` hace push automático de `EIP` (instruction pointer) en la pila y salta. `RET` hace pop de esa dirección y regresa. La pila es el mecanismo nativo, no un registro especial.

La Misma Operación, Tres Ensambladores: el Reto Real

Analizamos: `if (a < b) c = c + 1; else c = c - 1;` — parece trivial en C, pero revela diferencias profundas en cómo cada arquitectura modela la condicionalidad.

El problema central

MIPS y ARM difieren fundamentalmente en **condicionalidad por instrucción**: ARM permite que cada instrucción sea condicional; MIPS no, debiendo construir el booleano explícitamente y usar saltos.

Intel/x86 resuelve la comparación con `CMP` y usa banderas de estado (flags) para dirigir los saltos condicionales. Es el modelo más familiar para quienes vienen de lenguajes de alto nivel.

¿Qué queremos observar?

- ¿Cuántas instrucciones necesita cada arquitectura para el mismo `if/else`?
- ¿Dónde vive la "condición": en un registro, en flags de estado, o en sufijos de instrucción?
- ¿Cómo se accede a `c` si está en memoria?
- ¿El compilador puede optimizar el número de saltos?

Ejemplo Base en C: `if/else` con Enteros

Definimos el escenario exacto para la comparativa entre arquitecturas. Las suposiciones iniciales son parte del "contrato" entre compilador y hardware.

💻 Código C

```
if (a < b)
  c = c + 1;
else
  c = c - 1;
```

📌 Supuestos

a está en un registro. **b** está en un registro. **c** está en memoria (caso realista que obliga a load/store). El resultado vuelve a memoria.

🎯 Objetivo

Observar cómo cambian las instrucciones **de control** (comparación y saltos) y las **de memoria** (carga/almacenamiento) al traducir el mismo código fuente a MIPS, ARM y x86.

ⓘ Este ejemplo es ideal para el aula porque combina tres conceptos fundamentales: aritmética, control de flujo y acceso a memoria — todos en menos de 4 líneas de C.

MIPS: Comparar, Construir Condición y Ramificar

En MIPS no existe ejecución condicional por sufijo. El flujo de control siempre pasa por instrucciones de comparación explícitas seguidas de saltos condicionales.

Secuencia de instrucciones

```
        lw   $t2, 0($s2)    # c ← Mem[$s2]
        slt  $t0, $s0, $s1  # $t0 = (a < b) ? 1 :
0
        beq  $t0, $zero, else
# Bloque then: c = c + 1
        addu $t2, $t2, 1
        sw   $t2, 0($s2)    # Mem[$s2] ← c
        j    fin
else:   subu $t2, $t2, 1
        sw   $t2, 0($s2)
fin:
```

Puntos clave para el estudiante

- **Modelo Load/Store estricto**: primero `lw` para traer `c`, operación en registro, luego `sw` para guardar
- `slt` construye el booleano de la comparación en un registro
- `beq` / `bne` ramifican según ese valor
- La instrucción `j fin` es obligatoria para saltar el bloque `else`
- Todo es explícito: no hay operaciones implícitas ni flags ocultos

ARM: Lo Mismo, Pero con Ejecución Condicional "a Demanda"

ARM permite que muchas instrucciones se ejecuten condicionalmente usando sufijos. Esto reduce el número de saltos y puede mejorar el rendimiento del pipeline.

Secuencia de instrucciones

```
LDR   r2, [r5]        @ c ← Mem[r5]
CMP   r0, r1          @ Fijar flags: a - b
ADDLT r2, r2, #1      @ Si a
```

Puntos clave para el estudiante

- `CMP` actualiza los flags N, Z, C, V sin guardar resultado en registro
- `ADDLT` = "ADD si Less Than" — ejecuta solo si el flag corresponde
- `SUBGE` = "SUB si Greater or Equal" — el else sin instrucción de salto
- **Resultado pedagógico**: el `if/else` completo sin ningún salto (B)
- Menos instrucciones totales → menos penalización por predicción de saltos

Esta capacidad de ejecución condicional es exclusiva de ARM (y sus descendientes). Es una de las razones de su eficiencia energética.

Intel x86: CMP + Saltos por Flags (y Acceso Directo a Memoria)

x86 combina el modelo de flags con la capacidad de operar directamente sobre memoria, creando patrones más compactos pero con mayor complejidad de decodificación.

Secuencia de instrucciones

```
        CMP  eax, ebx        ; Flags ← a - b
        JGE  else            ; Salta si a >= b
; Bloque then: c = c + 1
        INC  DWORD [ecx]     ; Mem[ecx]++
        JMP  fin
else:   DEC  DWORD [ecx]     ; Mem[ecx]--
fin:
```

Puntos clave para el estudiante

- `CMP` calcula `a - b` y actualiza EFLAGS, sin guardar el resultado
- `JGE` lee el flag de signo/overflow para decidir el salto
- `INC DWORD [ecx]`: operación directamente sobre memoria — **no requiere load/store separado**
- Estilo CISC puro: una instrucción hace carga + operación + almacenamiento
- Más poderoso por instrucción, pero más complejo de decodificar internamente

Misma Idea: Traducir un `for` con Índice e Incremento

El bucle `for (i=0; i<N; i++) c[i] = a[i] + b[i];` combina control de flujo con acceso a arreglos. Es el ejemplo más completo para comparar arquitect

El desafío del acceso a arreglos

En MIPS y ARM, cada acceso a `a[i]`, `b[i]` y `c[i]` requiere calcular la dirección explícitamente: `base + i × 4`. El compilador genera instrucciones de escalado antes de cada carga o almacenamiento.

La ventaja de x86: modos de direccionamiento ricos

En Intel x86, el cálculo `[base + index*4]` puede expresarse directamente como un operando de memoria. No se necesita una instrucción separada para calcular la dirección.

El costo oculto: más instrucciones visibles en RISC

MIPS y ARM exponen cada micro-operación. Esto es educativamente valioso: el estudiante ve exactamente cuánto trabajo implica un simple `c[i] = a[i] + b[i]`.

MIPS: El Arreglo Obliga a "Sumar Direcciones + Load/Store"

Código MIPS del bucle

```
        add  $t0, $zero, $zero  # i = 0
loop:   slt  $t1, $t0, $s3      # i < N?
        beq  $t1, $zero, fin    # No → salir
        sll  $t2, $t0, 2        # offset = i*4
        add  $t3, $s0, $t2      # &a[i]
        lw   $t4, 0($t3)        # a[i]
        add  $t3, $s1, $t2      # &b[i]
        lw   $t5, 0($t3)        # b[i]
        add  $t6, $t4, $t5      # a[i]+b[i]
        add  $t3, $s2, $t2      # &c[i]
        sw   $t6, 0($t3)        # c[i] ← sum
        addi $t0, $t0, 1        # i++
        j    loop
fin:
```

Lecciones del laboratorio MIPS

- `sll $t2, $t0, 2`: multiplicar índice por 4 (tamaño de int) mediante desplazamiento lógico a la izquierda
- Tres cálculos de dirección distintos: uno para `a[i]`, uno para `b[i]`, uno para `c[i]`
- Cada carga y almacenamiento es una instrucción independiente — el load/store se hace evidente
- El contador `i` vive en un registro durante todo el bucle

Este bucle genera ~12 instrucciones por iteración en MIPS. Es un excelente ejercicio para calcular CPI (Cycles Per Instruction) del bucle completo.

ARM: El Bucle Se Siente Similar, Pero con Elegancia Adicional

Código ARM del bucle

```
        MOV  r4, #0              @ i = 0
loop:   CMP  r4, r6              @ i < N?
        BGE  fin                 @ No → salir
        LDR  r0, [r0, r4, LSL #2] @ a[i]
        LDR  r1, [r1, r4, LSL #2] @ b[i]
        ADD  r2, r0, r1          @ a[i]+b[i]
        STR  r2, [r2, r4, LSL #2] @ c[i]
        ADD  r4, r4, #1          @ i++
        B    loop
fin:
```

Diferencias clave respecto a MIPS

- `LDR r0, [r0, r4, LSL #2]`: el escalado del índice (×4) se integra en el operando de carga — no hay instrucción `SLL` separada
- La comparación usa `CMP` y flags directamente, sin construir un booleano en registro
- `BGE` (Branch if Greater or Equal) es más expresivo que el par `slt` + `beq` de MIPS
- Llamadas a funciones dentro del bucle: `BL` guarda retorno en `LR` automáticamente

Plantilla Binaria

Tipo	Instrucción	Plantilla binaria
R	add	000000 sssss ttttt ddddd 00000 100000
R	sub	000000 sssss ttttt ddddd 00000 100010
R	and	000000 sssss ttttt ddddd 00000 100100
R	or	000000 sssss ttttt ddddd 00000 100101
R	xor	000000 sssss ttttt ddddd 00000 100110
R	nor	000000 sssss ttttt ddddd 00000 100111
R	sll	000000 00000 ttttt ddddd aaaaa 000000
R	srl	000000 00000 ttttt ddddd aaaaa 000010
R	sra	000000 00000 ttttt ddddd aaaaa 000011
I	addi	001000 sssss ttttt iiiiiiiiiiiiiii
I	addiu	001001 sssss ttttt iiiiiiiiiiiiiii
I	andi	001100 sssss ttttt iiiiiiiiiiiiiii
I	ori	001101 sssss ttttt iiiiiiiiiiiiiii
I	xori	001110 sssss ttttt iiiiiiiiiiiiiii

UCO
Universidad Católica de Oriente
Vigilada Mineducación

Renovación Acreditación Institucional en ALTA CALIDAD



¡Gracias!